Mini project 2 general tips

Are boundaries included in model?

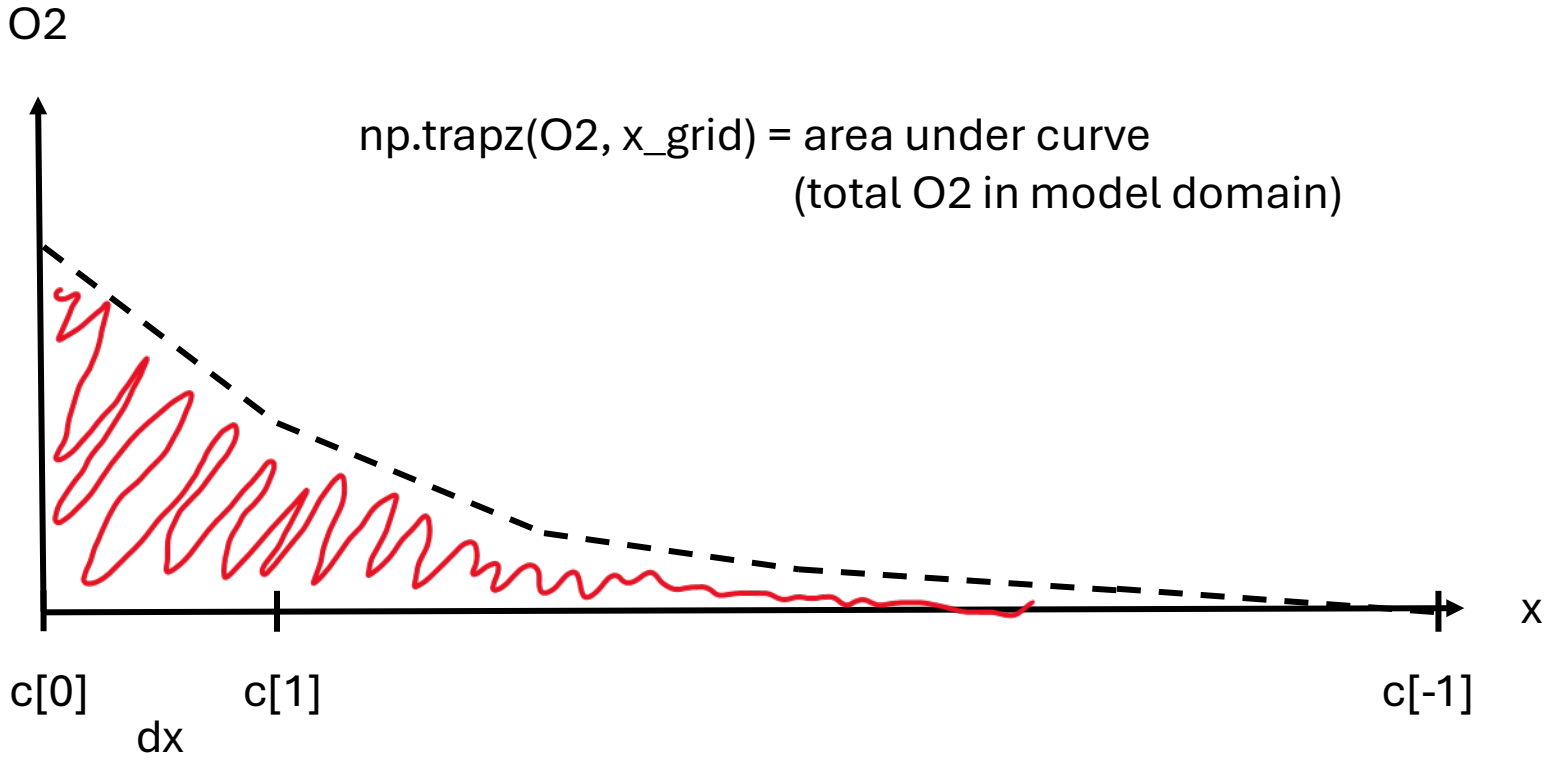
Dirichlet method 1: dcdt[0] = 0 and c0[0] = boundary value at edge of domain (included by dx/2)

Dirichlet method 2: dcdt[0] = D $\frac{c_{left} - 2c[0] + c[1]}{dx^2}$ and c0[0] = 0. Boundary value 1 dx outside domain.

Method 2 is shifted 1 dx left compared to method 1

Which method gives the highest concentrations in the model domain??

Calculating total mass in system

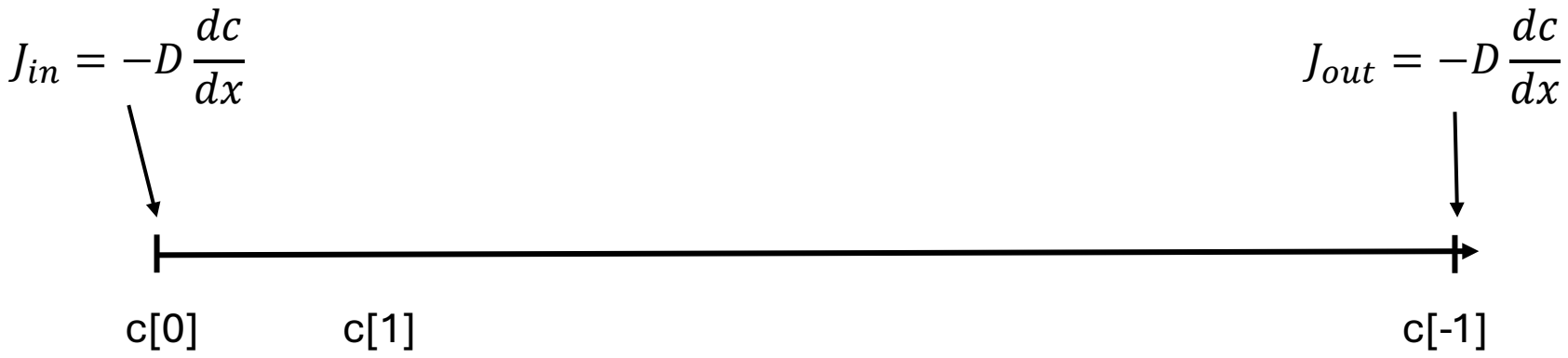


This should give the same as: np.sum(O2[1:-2] * dx) + np.sum(O2[0, -1] * dx/2)

Output is O2 mass unit / area unit.
If you want O2 in the whole tank → multiply by the surface area of the tank

Does calculating total mass in domain verify that you have model conservation?

Calculate flux in and flux out of domain → does it equal the mass in the system?



Flux is instant at some x position [g/m2/s]
Cumulative flux unit = [mass/area].

$J_{in\_cum} - J_{out\_cum} = accumulated\ in\ system\ without\ reaction$

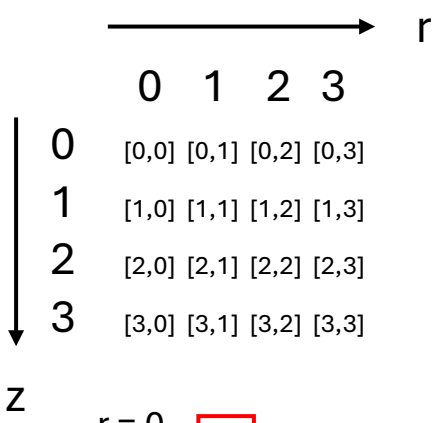
Cumulative flux → integrate fluxes with respect to time → np.trapz(fluxes, time)

Handling 2 state variables

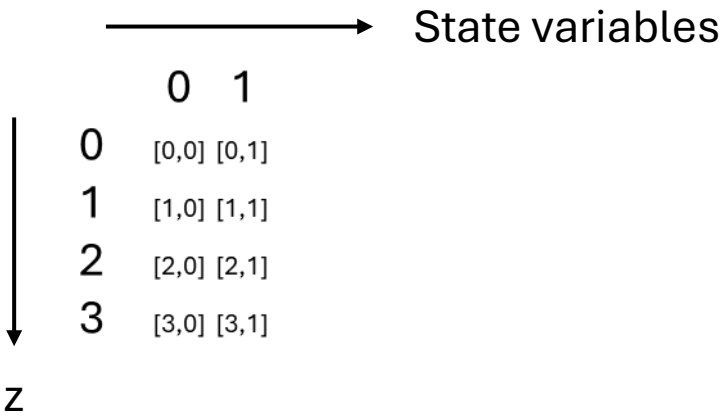
Flatten method was good for 2D arrays. It also works for 1D, but output is not that intuitive

Before flatten:

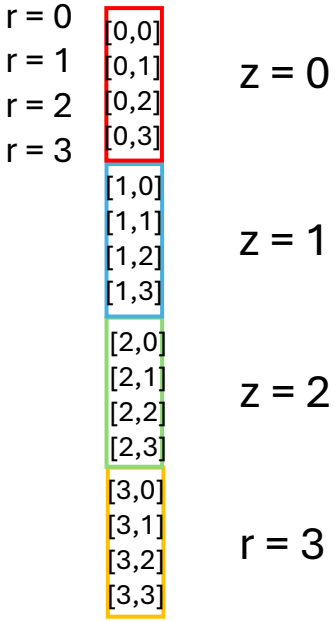
The 2D case

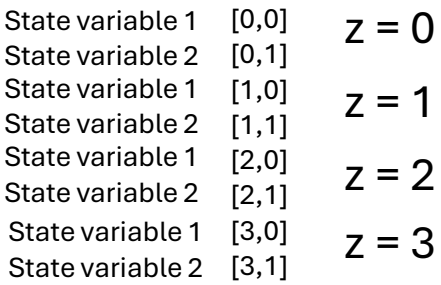


The 1D case with two state vars



After flatten:





Inspect and identify what rows and columns through the variable explorer to make sure you understand it.

Output from solve_ivp() gives time along the columns!

Alternative

See group exercise 6 solution under 09_23_sept

Use of np.concatenate()

O2 = np.zeros(len(x_grid)) = [0,0,0,0,0,0…]

OM = np.zeros(len(x_grid)) = [0,0,0,0,0,0…]

C0 = np.concatenate([O2, OM]) = [0,0,0,0,0,0…0,0,0,0,0,0…]

O2 OM

Output from solve_ivp()

State variable 1 z = 0
State variable 1 z = 1
State variable 1 z = 2
State variable 1 z = 3
State variable 2
State variable 2
State variable 2
State variable 2

Modular approach

Script running model

Load libraries
Load model
Call model with different input values
Plot model predictions

Model()

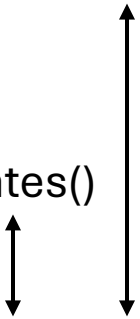
Rates()
Solve_ivp()
return dictionary

Difficult to debug

Script approach

Script with model

Load libraries
Define input values



Rates()

Solve_ivp()

Plot and inspect solve_ivp() output

Easy to debug